Київський національний університет імені Тараса Шевченка

Кафедра мережевих і Інтернет технологій

Лабораторна робота №3-4

з дисципліни

«Технології створення програмних продуктів»

ШАБЛОН PROTOTYPE ТА ШАБЛОН SINGLETON

Група: МІТ-м11

Студент: Переверзев Олексій

Київ 2026

Завдання 1: Шаблон Prototype

Використовуючи шаблон Prototype, реалізувати програму PCConfigurator, яка б дозволяла створювати екземпляри класів персональних комп'ютерів для домашнього, ігрового та офісного використання, із наперед заданими значеннями властивостей для характеристик основних компонентів: корпус, оперативна пам'ять, HDD, системна плата, процесор, відеоадаптер, монітор. Кожен компонент повинен мати обов'язкову характеристику Виробник та одну або більше специфічних саме для свого типу компонентів додаткові характеристики.

Код реалізації:

```
using System;
using System.Collections.Generic;

namespace PrototypeTask
{
    public abstract class Component //Prototype
    {
        public string Manufacturer { get; set; }
        public abstract Component Clone();
    }

    public class Case : Component //ConcretePrototype
    {
        public string FormFactor { get; set; }
        public override Component Clone() => (Component)this.MemberwiseClone();
    }

    public class RAM : Component
    {
        public int CapacityGB { get; set; }
        public int FrequencyMHz { get; set; }
        public override Component Clone() => (Component)this.MemberwiseClone();
    }

    public class HDD : Component
    {
        public int CapacityGB { get; set; }
        public string Type { get; set; }
        public override Component Clone() => (Component)this.MemberwiseClone();
    }

    public class Motherboard : Component
    {
        public string Socket { get; set; }
        public string Chipset { get; set; }
        public override Component Clone() => (Component)this.MemberwiseClone();
    }

    public class CPU : Component
    {
        public int Cores { get; set; }
        public double ClockSpeedGHz { get; set; }
        public override Component Clone() => (Component)this.MemberwiseClone();
    }

    public class VideoAdapter : Component
```

```
    {
        public int MemoryGB { get; set; }
        public override Component Clone() => (Component)this.MemberwiseClone();
    }

    public class Monitor : Component
    {
        public string Resolution { get; set; }
        public int RefreshRateHz { get; set; }
        public override Component Clone() => (Component)this.MemberwiseClone();
    }

    public class PersonalComputer
    {
        public string TypeName { get; set; }

        public Case CaseConfig { get; set; }
        public RAM RAMConfig { get; set; }
        public HDD HDDConfig { get; set; }
        public Motherboard MotherboardConfig { get; set; }
        public CPU CPUConfig { get; set; }
        public VideoAdapter VideoConfig { get; set; }
        public Monitor MonitorConfig { get; set; }

        public PersonalComputer Clone()
        {
            PersonalComputer clone = (PersonalComputer)this.MemberwiseClone();

            clone.CaseConfig = (Case)this.CaseConfig?.Clone();
            clone.RAMConfig = (RAM)this.RAMConfig?.Clone();
            clone.HDDConfig = (HDD)this.HDDConfig?.Clone();
            clone.MotherboardConfig = (Motherboard)this.MotherboardConfig?.Clone();
            clone.CPUConfig = (CPU)this.CPUConfig?.Clone();
            clone.VideoConfig = (VideoAdapter)this.VideoConfig?.Clone();
            clone.MonitorConfig = (Monitor)this.MonitorConfig?.Clone();

            return clone;
        }

        public void PrintSpecs()
        {
            Console.WriteLine($"\n=== Конфігурація: {TypeName} ===");
            Console.WriteLine($"Корпус: {CaseConfig.Manufacturer}, Форм-фактор:
{CaseConfig.FormFactor}");
            Console.WriteLine($"Мат. плата: {MotherboardConfig.Manufacturer}, Сокет:
{MotherboardConfig.Socket}, Чипсет: {MotherboardConfig.Chipset}");
            Console.WriteLine($"Процесор: {CPUConfig.Manufacturer}, Ядер:
{CPUConfig.Cores}, Частота: {CPUConfig.ClockSpeedGHz} GHz");
            Console.WriteLine($"ОЗП: {RAMConfig.Manufacturer}, Об'єм:
{RAMConfig.CapacityGB} GB, Частота: {RAMConfig.FrequencyMHz} MHz");
            Console.WriteLine($"Відеокарта: {VideoConfig.Manufacturer}, Пам'ять:
{VideoConfig.MemoryGB} GB");
            Console.WriteLine($"Накопичувач: {HDDConfig.Manufacturer}, Тип:
{HDDConfig.Type}, Об'єм: {HDDConfig.CapacityGB} GB");
            Console.WriteLine($"Монітор: {MonitorConfig.Manufacturer}, Роздільна
здатність: {MonitorConfig.Resolution}, {MonitorConfig.RefreshRateHz} Hz");
            Console.WriteLine("=====================================\n");
        }
    }

    public class PCConfigurator
    {
        private Dictionary<string, PersonalComputer> _prototypes = new
Dictionary<string, PersonalComputer>();
```

```
        public PCConfigurator()
        {
            InitializePrototypes();
        }

        private void InitializePrototypes()
        {
            _prototypes["Office"] = new PersonalComputer
            {
                TypeName = "Офісний ПК",
                CaseConfig = new Case { Manufacturer = "Chieftec", FormFactor =
"Mini-Tower" },
                MotherboardConfig = new Motherboard { Manufacturer = "ASUS", Socket
= "LGA1700", Chipset = "H610" },
                CPUConfig = new CPU { Manufacturer = "Intel", Cores = 4,
ClockSpeedGHz = 3.3 },
                RAMConfig = new RAM { Manufacturer = "Crucial", CapacityGB = 8,
FrequencyMHz = 2666 },
                VideoConfig = new VideoAdapter { Manufacturer = "Intel (Вбудована)",
MemoryGB = 1 },
                HDDConfig = new HDD { Manufacturer = "Kingston", Type = "SSD",
CapacityGB = 256 },
                MonitorConfig = new Monitor { Manufacturer = "Dell", Resolution =
"1920x1080", RefreshRateHz = 60 }
            };

            _prototypes["Home"] = new PersonalComputer
            {
                TypeName = "Домашній ПК",
                CaseConfig = new Case { Manufacturer = "Deepcool", FormFactor =
"Midi-Tower" },
                MotherboardConfig = new Motherboard { Manufacturer = "MSI", Socket =
"AM4", Chipset = "B550" },
                CPUConfig = new CPU { Manufacturer = "AMD", Cores = 6, ClockSpeedGHz
= 3.6 },
                RAMConfig = new RAM { Manufacturer = "Kingston", CapacityGB = 16,
FrequencyMHz = 3200 },
                VideoConfig = new VideoAdapter { Manufacturer = "NVIDIA", MemoryGB =
4 },
                HDDConfig = new HDD { Manufacturer = "Western Digital", Type =
"SSD", CapacityGB = 512 },
                MonitorConfig = new Monitor { Manufacturer = "LG", Resolution =
"1920x1080", RefreshRateHz = 75 }
            };

            _prototypes["Gaming"] = new PersonalComputer
            {
                TypeName = "Ігровий ПК",
                CaseConfig = new Case { Manufacturer = "NZXT", FormFactor = "Midi-
Tower" },
                MotherboardConfig = new Motherboard { Manufacturer = "Gigabyte",
Socket = "AM5", Chipset = "X670" },
                CPUConfig = new CPU { Manufacturer = "AMD", Cores = 8, ClockSpeedGHz
= 4.5 },
                RAMConfig = new RAM { Manufacturer = "G.Skill", CapacityGB = 32,
FrequencyMHz = 6000 },
                VideoConfig = new VideoAdapter { Manufacturer = "NVIDIA", MemoryGB =
12 },
                HDDConfig = new HDD { Manufacturer = "Samsung", Type = "NVMe SSD",
CapacityGB = 2000 },
                MonitorConfig = new Monitor { Manufacturer = "ASUS ROG", Resolution
= "2560x1440", RefreshRateHz = 165 }
            };
        }
```

```
        public PersonalComputer CreatePC(string type)
        {
            if (_prototypes.ContainsKey(type))
            {
                return _prototypes[type].Clone();
            }
            throw new ArgumentException("Невідомий тип ПК");
        }
    }

    class Program
    {
        static void Main(string[] args)
        {
            PCConfigurator configurator = new PCConfigurator();

            PersonalComputer myOfficePC = configurator.CreatePC("Office");
            PersonalComputer myGamingPC = configurator.CreatePC("Gaming");
            PersonalComputer myHomePC = configurator.CreatePC("Home");

            myOfficePC.PrintSpecs();
            myGamingPC.PrintSpecs();
            myHomePC.PrintSpecs();

            Console.WriteLine("\n--- Модернізуємо створений офісний ПК (додаємо ОЗП
та міняємо монітор) ---");
            myOfficePC.RAMConfig.CapacityGB = 16;
            myOfficePC.MonitorConfig.Manufacturer = "Samsung";
            myOfficePC.PrintSpecs();

            Console.WriteLine("\n--- Створюємо ще один базовий офісний ПК з
прототипу ---");
            PersonalComputer anotherOfficePC = configurator.CreatePC("Office");
            anotherOfficePC.PrintSpecs();

            Console.ReadKey();
        }
    }
}
```

Результат виконання програми:

```
=== Конфігурація: Офісний ПК ===
Корпус: Chieftec, Форм-фактор: Mini-Tower
Мат. плата: ASUS, Сокет: LGA1700, Чипсет: H610
Процесор: Intel, Ядер: 4, Частота: 3,3 GHz
ОЗП: Crucial, Об'єм: 8 GB, Частота: 2666 MHz
Відеокарта: Intel (Вбудована), Пам'ять: 1 GB
Накопичувач: Kingston, Тип: SSD, Об'єм: 256 GB
Монітор: Dell, Роздільна здатність: 1920x1080, 60 Hz
====================================


=== Конфігурація: Ігровий ПК ===
Корпус: NZXT, Форм-фактор: Midi-Tower
Мат. плата: Gigabyte, Сокет: AM5, Чипсет: X670
Процесор: AMD, Ядер: 8, Частота: 4,5 GHz
ОЗП: G.Skill, Об'єм: 32 GB, Частота: 6000 MHz
Відеокарта: NVIDIA, Пам'ять: 12 GB
Накопичувач: Samsung, Тип: NVMe SSD, Об'єм: 2000 GB
Монітор: ASUS ROG, Роздільна здатність: 2560x1440, 165 Hz
====================================


=== Конфігурація: Домашній ПК ===
Корпус: Deepcool, Форм-фактор: Midi-Tower
Мат. плата: MSI, Сокет: AM4, Чипсет: B550
Процесор: AMD, Ядер: 6, Частота: 3,6 GHz
ОЗП: Kingston, Об'єм: 16 GB, Частота: 3200 MHz
Відеокарта: NVIDIA, Пам'ять: 4 GB
Накопичувач: Western Digital, Тип: SSD, Об'єм: 512 GB
Монітор: LG, Роздільна здатність: 1920x1080, 75 Hz
====================================
```

```
--- Модернізуємо створений офісний ПК (додаємо ОЗП та міняємо монітор) ---

=== Конфігурація: Офісний ПК ===
Корпус: Chieftec, Форм-фактор: Mini-Tower
Мат. плата: ASUS, Сокет: LGA1700, Чипсет: H610
Процесор: Intel, Ядер: 4, Частота: 3,3 GHz
ОЗП: Crucial, Об'єм: 16 GB, Частота: 2666 MHz
Відеокарта: Intel (Вбудована), Пам'ять: 1 GB
Накопичувач: Kingston, Тип: SSD, Об'єм: 256 GB
Монітор: Samsung, Роздільна здатність: 1920x1080, 60 Hz
=====================================


--- Створюємо ще один базовий офісний ПК з прототипу ---

=== Конфігурація: Офісний ПК ===
Корпус: Chieftec, Форм-фактор: Mini-Tower
Мат. плата: ASUS, Сокет: LGA1700, Чипсет: H610
Процесор: Intel, Ядер: 4, Частота: 3,3 GHz
ОЗП: Crucial, Об'єм: 8 GB, Частота: 2666 MHz
Відеокарта: Intel (Вбудована), Пам'ять: 1 GB
Накопичувач: Kingston, Тип: SSD, Об'єм: 256 GB
Монітор: Dell, Роздільна здатність: 1920x1080, 60 Hz
=====================================
```

У завданні реалізовано шаблон Prototype з використанням підходу глибокого клонування (Deep cloning). Клас PersonalComputer не лише копіює себе, а й викликає метод Clone() для кожного свого компонента. Це дозволяє створювати нові незалежні об'єкти на основі наперед заданих еталонів ("Офісний", "Домашній", "Ігровий"), не перевантажуючи систему викликами конструкторів із великою кількістю параметрів.

Завдання 2: Шаблон Singleton

Реалізуйте клас Service, який буде використовувати шаблон Singleton для створення єдиного об'єкта сервісу. Клас Service повинен мати метод DoWork(), який приймає рядок в якості параметра і виводить його на консоль. Клас Service також повинен мати метод GetStatus(), який повертає рядок «Working» або «Idle» в залежності від того, чи виконується в даний момент метод DoWork() чи ні.

Код реалізації:

```
using System;
using System.Threading;

namespace SingletonTask
{
    public sealed class Service
    {
        private static readonly Lazy<Service> _instance = new Lazy<Service>(() =>
new Service());

        private bool _isWorking = false;

        public static Service Instance
        {
            get { return _instance.Value; }
        }

        private Service() { }
```

```
        public string GetStatus()
        {
            return _isWorking ? "Working" : "Idle";
        }

        public void DoWork(string message)
        {
            _isWorking = true;
            Console.WriteLine($"[Поточний статус: {GetStatus()}]");

            Console.WriteLine($"-> Виконується завдання: {message}");

            Thread.Sleep(1500);

            _isWorking = false;
            Console.WriteLine($"[Поточний статус: {GetStatus()}] Завдання
завершено.\n");
        }
    }

    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("=== Демонстрація шаблону Singleton ===\n");

            Service myService = Service.Instance;

            Console.WriteLine($"Статус до початку роботи:
{myService.GetStatus()}\n");

            myService.DoWork("Діагностика зникнення лінку на комутаторі агрегації");

            Service anotherServiceReference = Service.Instance;

            if (myService == anotherServiceReference)
            {
                Console.WriteLine("Перевірка: myService та anotherServiceReference –
це один і той самий об'єкт!\n");
            }

            anotherServiceReference.DoWork("Оновлення прошивки на клієнтському
MikroTik");

            Console.WriteLine($"Статус після завершення всіх робіт:
{myService.GetStatus()}");

            Console.ReadKey();
        }
    }
}
```

Результат виконання програми:

```
=== Демонстрація шаблону Singleton ===

Статус до початку роботи: Idle

[Поточний статус: Working]
-> Виконується завдання: Діагностика зникнення лінку на комутаторі агрегації
[Поточний статус: Idle] Завдання завершено.

Перевірка: myService та anotherServiceReference – це один і той самий об'єкт!

[Поточний статус: Working]
-> Виконується завдання: Оновлення прошивки на клієнтському MikroTik
[Поточний статус: Idle] Завдання завершено.

Статус після завершення всіх робіт: Idle
```

Реалізовано потокобезпечний "лінивий" (Lazy) Singleton. Конструктор класу Service приховано (private), а створення об'єкта відбувається лише при першому зверненні до глобальної точки доступу Instance. Це гарантує наявність лише одного екземпляра сервісу в оперативній пам'яті, що доведено в клієнтському коді перевіркою посилань на ідентичність.

Висновки: Під час виконання роботи було досліджено патерни, що породжують об'єкти, які допомагають оптимізувати виділення пам'яті та керування життєвим циклом сутностей: Prototype довів свою ефективність при конструюванні складних об'єктів (таких як ПК). Глибоке клонування дозволяє швидко отримувати нові незалежні екземпляри на основі еталонів, уникаючи рутинного налаштування через розгалужену ієрархію підкласів. Singleton забезпечує надійну архітектуру для сервісних класів, надаючи єдину глобальну точку доступу до ресурсів та спільного стану (як-от відстеження статусу Working/Idle), виключаючи ризик створення дублікатів.